Field Hours

CSCE 701 Foundations of Cybersecurity

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 40 minutes

Threats, including machines and people

Security is a claim about assets, adversaries, and what they must not achieve. An asset is something you would notice losing: a key, a file, the ability to stop a pump, a person's private data. An adversary is a concrete party with a budget and a position: on the network, in the building, or already running code on the machine. A threat model lists the assets, the adversaries, and the actions you refuse them. A product that says it is secure and does not list these is not making a claim.

Confidentiality means the adversary does not learn the asset. Integrity means they do not change it without detection. Availability means they do not take it away. Most systems pick two and hand-wave the third. A lock on a disk does nothing for availability if the disk is in a fire. Write the three words next to each asset and mark the ones you are not covering.

Cyber-physical security is the same model when the asset is a physical process. A false sensor reading is an integrity failure. A valve that will not close is an availability failure. The computer is not the asset. The computer is the path. A threat model that stops at the login page misses the pump.

People are part of the path. A password reset done over a phone call is an authentication protocol whose adversary is a convincing voice. Social engineering is not a footnote after the cryptography. It is a protocol with weak binding between the speaker and the identity. Put it in the model in the first lecture or you will design a perfect channel to a person who should not be on it.

A claim you can check. Each layer is a sentence, not a product category.

What must not happen

Who is trying

What the asset is

What you are not covering

A small clinic's records

1. Asset: the visit notes. Adversary: a person on the public network, and a curious staff account.

2. Refuse the network adversary the ability to read or alter notes. Refuse the staff account the ability to read notes for patients they are not treating.

3. You are not, in this model, stopping a staff member who is treating the patient from copying notes out by hand. Say so. That limit is part of the model.

Problem 1. The clinic encrypts the disk and leaves visit notes in a world-readable file on the server. Which property fails against the network adversary?

Answer. Confidentiality. Disk encryption does not cover a file the server hands out. The asset moved to a path the control does not see.

Problem 2. A water plant's model lists the login password and not the valve. What asset did they skip?

Answer. The physical process: whether the valve obeys a true command. A stolen password matters because of the valve, and a forged command on the control network matters even with a fine password on the office door.

Lecture 2 · 40 minutes

Memory safety and injection

A buffer that writes past its end is an integrity failure in the threat model. The adversary who controls the length or the bytes can change a return address or a function pointer and run their own code. The bug is not that the language has pointers. The bug is that the program believed a length the adversary chose. Bounds checks, or a language that inserts them, close this path. A compiler warning you turned off does not.

Injection is the same shape in a parser. The program builds a command, a query, or a page by gluing a string the adversary wrote into a string the program wrote. The adversary's string closes the intended grammar and opens a new one. A quote in a query, a semicolon in a shell command, a tag in a page. The repair is to keep data out of the grammar: a parameter slot the parser cannot reinterpret, or an encoder that makes the data grammatically inert.

Authentication sits on these bugs. A login check that is correct and a query that is injectable is an open door with a good lock pictured beside it. The adversary does not have to pass the check if they can change the query that implements the check. Read the constructed string. If any character in it came from the request, you are in this lecture.

Firewalls do not fix a bug in your parser. A firewall drops packets you told it to drop. The injectable query arrives as ordinary traffic to a port you left open because the application needs it. The control belongs in the code that builds the query. Defense in depth is a second control, not a substitute for the first.

Two bugs, one shape. The adversary supplied a structure the program treated as data.

A long buffer

Length is the weapon

Writes past the end

Becomes code

A crafted string

Punctuation is the weapon

Closes the grammar

Becomes a command

A login query built by gluing

1. The program builds a query that asks for a user whose name equals the request's name field.

2. The request contains a name that closes the quote and adds a clause the parser understands as 'or true'.

3. The query returns rows. The login succeeds. The password check never constrained the adversary, because the query was no longer the query you wrote.

Problem 1. You switch the query to a parameter slot, and the same bytes arrive in the name field. Why does the login not succeed?

Answer. The bytes stay inside one value. The parser does not see a quote or a second clause. It sees a strange username, finds no row, and rejects the login.

Problem 2. A firewall blocks all ports except the application's. Does that stop the injectable query?

Answer. No. The attack is a legal request to the open port. The firewall is doing what you asked. The application is still concatenating strings.

Lecture 3 · 45 minutes

Symmetric cryptography

A cipher hides a message from someone who does not have the key. Symmetric means the same key encrypts and decrypts. The one-time pad xors the message with a key as long as the message, used once. It is information-theoretically hiding if the key is random and never reused. Reuse is fatal: the xor of two ciphertexts is the xor of the two plaintexts, and the key cancels. A pad you reused is not a pad. It is a mistake with a famous name.

A stream cipher is a pad you generate from a short key and a nonce. The nonce must not repeat under the same key. A block cipher encrypts a fixed-size block. Modes extend it to longer messages. A mode that is a fancy name for 'xor a repeated block' repeats the pad and fails in the same way. An authenticated mode tells you if the ciphertext was modified. Encryption without authentication does not, and an adversary who can flip bits may flip a bit in a known place.

Keys are assets. A key in a source repository is a key the history will keep. A key in a binary you ship is a key the user has. Deriving a key from a password is a different, slower function, because passwords are guessable. A cipher does not make a weak password strong. It makes the guessing happen at the password check.

Cryptography does not decide who may speak. It decides what someone without the key can learn or alter. If the threat model includes a staff member who has the key, symmetric encryption of the database does not address that staff member. You already wrote that limit in lecture one. Do not let a cipher talk you out of it.

A stream cipher. The nonce is not a secret. Reusing it under the same key is the failure.

Key and nonce -> Keystream -> Xor message -> Ciphertext

A pad used twice

1. Plaintext one is the bits 1010. Plaintext two is 1100. The key is 0110.

2. Ciphertext one is 1100. Ciphertext two is 1010.

3. Xor the ciphertexts: 0110, which is also the xor of the plaintexts. The key cancelled. Anyone who guesses one plaintext now has the other.

Problem 1. A protocol uses a block cipher and no authentication. The adversary flips a bit in the ciphertext. What can you say when you decrypt?

Answer. You get some other plaintext, and you have no cryptographic signal that it is not the original. Integrity was not provided. Use an authenticated mode, or attach a tag you verify before you trust the bytes.

Problem 2. Why is a one-time pad key the length of one message unsafe for a second message?

Answer. Xor of the two ciphertexts equals xor of the two plaintexts. The key drops out. That is enough to recover language-like messages and exact known pieces. A new key, or a nonce that makes the keystream new, is required.

Lecture 4 · 40 minutes

Hashes, tags, firewalls, and detection

A cryptographic hash maps a message to a short digest. A collision is two messages with the same digest. A preimage is a message for a digest you were given. A password hash must be slow and salted so a stolen file is not a dictionary you can check at cipher speed. A hash of a file detects accidents. It does not detect an adversary who can change the file and the hash together. For an adversary you need a key.

A message authentication code is a keyed hash. The receiver recomputes it and rejects the message on a mismatch. The key is what turns a hash into an integrity check against an adversary. Order matters in the construction: a homemade 'hash the message and the key' can be weaker than the construction you were told to use. Use the function your library calls a MAC, and compare tags in constant time so the comparison does not leak how many bytes matched.

A firewall is a policy on packets: these addresses, these ports, drop or allow. It is a coarse control and a useful one. It does not see an attack inside an allowed protocol. An intrusion detection system watches for known bad patterns or for behavior that is rare. A known pattern misses a new attack. A rare-behavior alarm rings when the world changes for a benign reason. Both tools produce a log. A log nobody reads is not a control.

Authentication of a person is a separate mechanism: a secret, a device, or a biometric, and preferably more than one. The MAC authenticates a message between keys you already placed. It does not decide that the human at the keyboard is the human you hired. Put the human check at the start of the session. Put the MAC on the bytes after that.

A hash and a MAC answer different adversaries. The difference is the key.

Hash

No key

Catches accidents

An editor can rehash

MAC

Shared key

Catches modification

The editor lacks the key

A file and its digest

1. You publish a file and a hash of the file on the same site.

2. An adversary replaces both with a new file and a new hash.

3. A visitor who trusts the site sees a match. The hash did not help. A signature under a key the visitor already has, or a MAC under a key that was exchanged earlier, is the check that fails.

Problem 1. Password hashes are salted and slow. What does the salt stop, and what does the slowness stop?

Answer. The salt stops one precomputed table from matching every user who chose the same password. The slowness stops the adversary from trying millions of guesses per stolen hash per second. You want both.

Problem 2. An intrusion system alerts on every new source address, and the office changed networks this morning. What is wrong with treating every alert as an incident?

Answer. The detector has no model of a benign change, so the morning is a pile of alerts and the real one is hidden. Tune the detector, or require a second signal. A log of everything is not detection.

Lecture 5 · 45 minutes

Public keys and what a certificate checks

Public-key encryption uses a pair. The public key encrypts or verifies. The private key decrypts or signs. You can publish the public key. You must not publish the private key. A signature is the private key applied to a hash of the message. Anyone with the public key can check it. A signature is not encryption. It does not hide the message. It binds the message to the holder of the private key, if you trust that the public key belongs to the party you think it does.

That last 'if' is the certificate. A certificate is a signature by someone you already trust, over someone else's public key and name. Checking a certificate means checking the signature, the name, the dates, and whether the key was revoked. Skipping the name check means you talked to someone who has a valid certificate for a different name. The cryptography succeeded. The authentication failed.

A session on the public network typically agrees on a symmetric key using the public-key machinery, then switches to the symmetric cipher from lecture three because it is faster. The handshake's job is to make sure the symmetric key is shared with the named party and not with the adversary in the middle. If the client does not check the name, the adversary in the middle is the named party as far as the code is concerned.

Social attacks bypass the handshake. A person who is talked into reading a code to the wrong caller, or into installing a new root of trust, has moved the 'someone you already trust'. No signature detects that. The threat model from lecture one has to include it, and the recovery is a procedure: who may add a root, how a password reset is bound to a person, what you do when a key leaks. Cryptography ends. The procedure is the rest of the

control.

A certificate check. Stop at the first failure. A valid signature for the wrong name is a failure.

1. Check the signature

2. Check the dates and revocation

3. Check the name

4. Then use the key

A certificate for the wrong host

1. You meant to reach mail.example. The peer offers a certificate whose name is other.example, signed by a trusted issuer, in date.

2. The signature verifies. The name does not match.

3. A client that ignores the name will set up a session with whoever offered the certificate. The issuer did their job. The client did not.

Problem 1. Does a signature hide the message from an eavesdropper?

Answer. No. A signature is computed on the message, and the message is sent beside it. Hiding requires encryption. You often want both, and they are different operations.

Problem 2. A caller asks an employee to read out a one-time code from the login device. Which control failed?

Answer. The binding between the code and the real session. The code is a secret the employee handed to an unverified voice. The certificate on the website never saw the call. The procedure for what staff may read aloud is the missing control.

Lecture 6 · 180 minutes

Authentication

Authentication is how a system decides that a request comes from a principal it has seen before. The common secret is a password. The system must not store the password. It stores a slow hash of the password plus a unique salt. A fast hash lets an attacker try a dictionary against every user at once if the salts are missing, and against one user quickly even if you salted a fast hash.

A salt is not a pepper and not encryption. It is a unique value stored beside the hash so that two users with the same password do not have the same stored value, and so a precomputed table does not apply to everyone. The salt is not secret. The password is.

Session identifiers after login are capabilities. They must be unguessable, transported only over the secure channel, and revocable. A session id in a URL will leak through logs and referrers. A session that never expires on a shared computer is a gift to the next person who sits down.

Multi-factor authentication adds a second check that is not the same secret. A second password is not a second factor. A stolen password database should not be enough. Phishing can still sit in front of both factors if the user

will type them into the wrong page. The name check on the certificate is part of authentication.

What is stored.

A unique salt, not secret

A slow hash of salt and password

Not the password

Not a fast hash of the password alone

Two users, one password

1. Ada and Ben both choose the same password. Without salts, the stored hashes match, and a dictionary attack cracks both at once.

2. With unique salts, the stored values differ. The attacker recomputes the hash per user.

3. If the hash is fast, per-user recomputation is still cheap. The slow hash is what makes the dictionary expensive. Salt and speed solve different problems.

Problem 1. You encrypt passwords with a key stored on the same server. Why is that worse than a slow hash?

Answer. A thief who copies the database and the key decrypts every password. A hash has no key that reverses it. Encryption is reversible by design. Password storage must not be.

Problem 2. A session id is the user's email address. What can another user do?

Answer. Present that email address and be treated as the victim, if the server accepts it as the session. The id has to be unguessable. An identifier you can look up is not a session secret.

Lecture 7 · 180 minutes

Access control

Authentication says who is calling. Access control says what they may do. An access-control list on an object lists principals and actions. A capability is an unforgeable reference that itself confers the action. The operating-systems week introduced both. Here they apply to records in your system, not only to files.

The confused deputy is a program with authority that uses it on the attacker's behalf. A report generator that can read every record, and that accepts a path from the user, will read the payroll file if the user names it. The generator's authority and the user's authority were mixed. Pass a capability for the records the user may see, not a filename and a powerful process.

Default deny is the policy you can audit. Default allow is how a new endpoint ships without a check. List the endpoints and the rule on each. An endpoint missing from the list is a bug.

Privilege separation splits the program so the part that parses untrusted input does not hold the database credentials. A bug in the parser then cannot, by itself, dump the table. The split is a process boundary or a careful module boundary with no shared secret. A comment that says 'do not call this' is not separation.

Two designs for the same report.

Deputy

Process can read all files

User supplies a path

The payroll file is reachable

Capability

User holds a handle to their reports

The process cannot name the payroll file

A bad path is not a name the process can open

A path from the user

1. The report tool runs as a service account that can read every customer's files. It takes a path from the query string.

2. The attacker asks for the path of another customer's export. The tool's check, if it only checks that the file exists, will read it.

3. The fix is to derive the path from the authenticated principal, or to require a capability issued to that principal. Do not concatenate the user's string onto a directory and call it access control.

Problem 1. Why is 'we check the role in the UI' not access control?

Answer. The UI is not the boundary. A client can send the request the UI would have hidden. The check has to run on the server that holds the data, on every request.

Problem 2. A new endpoint ships with no rule, and the framework allows the call. What is the default, and why is it wrong?

Answer. The default is allow. Any authenticated, or unauthenticated, caller can use the endpoint. Default deny means a missing rule is a refusal, so the endpoint stays dark until someone writes the rule.

Lecture 8 · 180 minutes

The network attacker

The attacker on the path can read, drop, reorder, and replay anything that is not inside a secure channel. The networks course built the channel. This week is the attacks that remain when you forget a piece. Replaying a captured request is the first. A unique nonce or a timestamp checked at the server makes the replay fail. Without it, a copied 'pay' request pays twice, which you have now seen from three courses at once.

Spoofing a source address is how a packet claims to be from someone else, on a protocol that does not authenticate. UDP and raw IP do not. TCP's handshake makes off-path spoofing harder and does not stop an on-path attacker. Do not treat a source address as an identity.

A downgrade attack strips the secure option from a handshake that is willing to proceed without it. Refuse to proceed. The earlier lecture said this. Here it is the attacker's move, named.

DNS lies are a network attack before you ever connect. If the resolver is poisoned, you open a secure channel to the wrong host, and the certificate check is the remaining defense. If you skip the name check, the lie works.

A replay.

1. Capture a valid request

2. Send it again

3. A nonce the server remembers

4. The second send fails

A copied payment

1. The attacker records a request that was correctly signed, including the body.

2. They send it again. The signature still verifies. The server charges again if it has no memory of the request id.

3. The server stores the id for longer than the signature's lifetime. The second request fails the replay check even though the bytes are perfect.

Problem 1. Why is a source IP address not an authenticated identity?

Answer. Off the path, some protocols let the sender write any source they want. On the path, the attacker does not even need to spoof. Identity comes from a key or a password checked at the endpoint, not from the address on the packet.

Problem 2. A signature covers the body and not the request id. What does the attacker replay?

Answer. The body, with a fresh id if the server only checks the id for uniqueness and the signature does not bind the id. Bind every field that matters, including the id and the timestamp, or the signature is on the wrong message.

Lecture 9 · 180 minutes

The browser

A web session adds attacks that are not on the packet path. Cross-site scripting runs the attacker's script in the victim's origin, which holds the session. The defense is to encode untrusted data on the way out, and to not splice untrusted data into script. A sanitizer you half-wrote is how the bug survives. Use a maintained encoder and a content security policy as the backstop, not as the only control.

Cross-site request forgery tricks the victim's browser into sending the victim's cookies to your site. A cookie that is sent on every request is the vulnerability. A token the attacker's page cannot read, checked on the state-changing request, is the fix. Same-site cookies are a strong default and not a theory you should skip.

The origin is the scheme, the host, and the port. A subdomain is a different origin. A script that treats them as the same will share a secret across more parties than you listed. Write the origins down.

Dependencies in the page are code you run. A third-party script can read the session if it is in your origin. Load it only if you would give that party a copy of the database.

Two browser attacks.

Script injection

Attacker's script runs in your origin

Encode output

Do not build script from input

Forged request

Victim's browser sends cookies

Attacker's page cannot read a token

Check the token on the write

A name in a page

1. A user named themselves <script>.... The page inserts the name into HTML without encoding.

2. The browser executes the script in your origin and can read the session.

3. Encoding the name as text removes the tag. The script does not run. A test with that name belongs in the suite. The software-engineering regression rule applies.

Problem 1. Your CSRF token is in a cookie and nowhere else, and the browser sends cookies automatically. Does the check work?

Answer. No. The attacker's request will include the cookie. The token has to be somewhere the attacker's page cannot force the browser to send, typically a header or a field that script on another origin cannot set on your request. A cookie alone is the credential being forged, not the proof.

Problem 2. A third-party analytics script is included in the payment page. What can it do?

Answer. Whatever your own script can do, if it runs in your origin: read the DOM, including the account, and send it elsewhere. Treat it as a deputy with full page access. If that is unacceptable, do not load it there.

Lecture 10 · 180 minutes

Failures of the mechanism

Cryptography fails in use more often than it fails in the primitive. Reusing a nonce in a stream cipher, as the earlier lecture showed with a pad used twice, is a use failure. So is trusting a certificate you did not check, truncating a MAC, and encrypting without authenticating. The primitive can be excellent and the composition wrong.

Side channels leak a secret through time, power, or errors. A comparison that returns early on the first mismatched password byte tells an attacker which prefix they have right. Compare in constant time when the secret is a MAC or a password hash check that an attacker can time. This is not a complete treatment of side channels. It is the case you can introduce by writing a normal loop.

Randomness is a dependency. A key generated from a clock, or from a random function that is not a cryptographic generator, is guessable. Use the operating system's cryptographic generator. Do not seed your own from the time.

Errors must not reveal which check failed when that information is a password oracle. 'No such user' and 'wrong password' as distinct messages let an attacker enumerate accounts. One message for both is the usual fix. The log on the server can still tell them apart.

A MAC check that leaks.

Compare byte 0 and return

The attacker times the return

A correct prefix takes longer

Compare the whole tag in constant time

A pad used twice, recalled

1. Two messages encrypted with the same one-time pad are revealed by their XOR, up to the language you assume.

2. A stream-cipher nonce reused under one key is the same bug with a modern name.

3. The fix is a unique nonce per message under that key, stored or counted, not hoped for. A random nonce from a small space will collide. Size the space.

Problem 1. Password check returns in 1 millisecond if the first character is wrong and 2 milliseconds if it is right. What is the attacker learning?

Answer. The first character, by timing, then the next, the same way. The early return is an oracle. Compare the entire hash or the entire MAC before returning, in time that does not depend on the mismatch position.

Problem 2. The login form says 'unknown user' or 'bad password.' Why does that matter if passwords are hashed?

Answer. It confirms which addresses have accounts, which is the list the attacker needed before the hashes, or instead of them. The hash does not fix the oracle. Use one error for both cases.

Lecture 11 · 180 minutes

Logs and incidents

An incident is a claim that a security property failed, plus the evidence. The evidence is logs. A log that stores passwords or full card numbers is a second breach waiting. Log the request id, the principal, the action, the result, and the time. Log enough to detect, not a copy of the database.

Detection is a query you can run. Five failed logins followed by a success from a new address is a query. A terabyte of unstructured text is not. The alert rule from the software-engineering course applies: if nobody will act, it is not detection.

When you suspect a stolen session, you revoke the session and the refresh tokens, fix the hole that stole it, and decide whether passwords must be reset. Resetting every password because a CSS bug leaked a page title is a different incident from a stolen password database. Match the response to the evidence.

Write the timeline from the logs after a drill, not only after a breach. A drill that restores from backup tells you whether the backup exists. A backup you have not restored is a file.

A small incident.

1. A query that fires

2. A timeline from the logs

3. Revoke what was taken

4. A test that would catch the hole

A stolen session

1. The log shows a session used from two countries in five minutes. The user was not traveling. You revoke the session.

2. The timeline shows the session cookie appeared in a referrer log because it was in a URL. That is the hole.

3. You move the id out of the URL, add a regression test that the cookie is not in the query string, and you do not reset every password if the passwords were not in the log. The evidence did not say they were.

Problem 1. The application logs the full Authorization header. What did you create?

Answer. A second copy of the bearer token, visible to anyone who can read the logs. Log that a credential was present, not the credential. Treat the log store as part of the trusted computing base or strip it.

Problem 2. A detector fires only on a signature you wrote after the last incident. What class of attack does it miss?

Answer. The next one that does not match that signature. Signature detection catches repeats. A control that removes the hole, plus a test, is the repair. The detector is the alarm, not the fix.

Lecture 12 · 180 minutes

The threat model, written

The first lecture asked for a threat model of a small clinic. This week you write one for the capstone, one page. Assets, adversaries, what they can do, and what you explicitly ignore. Ignoring a national intelligence service is allowed if you say so. Ignoring a stolen laptop, if the capstone holds personal records, is not.

For each asset, name the control: the hash, the lock, the channel, the check. A control you cannot point to in a design note is a slogan. The page should be checkable.

Include one abuse case the way the software course included a failing test. The attacker replays a request, injects

a query, or fetches another user's path. Say which control stops it.

This page is part of the security course's hours. It is also an entrance ticket to the capstone. You will not bolt it on after the code.

One page.

Assets

Adversaries and what they can do

What you ignore, said aloud

A control for each asset

One abuse case

The clinic, revisited

1. Asset: the record. Adversary: another patient, and a thief who copies the disk.

2. Controls: access check on each read, disk encryption or a database that demands a credential, a session that is not in the URL.

3. Ignored: a coerced insider with a legitimate login, if you say you are not solving that this term. Unsaid, that omission looks like a hole. Said, it is a scope.

Problem 1. The model says 'we use encryption' and does not say which asset or which key. What is wrong?

Answer. Encryption is not a control until you name the data, the key, and who can decrypt. 'We use encryption' cannot be reviewed. Name the column or the channel and the key store.

Problem 2. You ignore denial of service. What must the page still say if the capstone takes payments?

Answer. That a flood can stop payments, and that this is accepted for the exercise or mitigated by a specific limit. Silence means you did not decide. A payment system that falls over may create its own integrity bugs if retries are wrong. Point at the retry control even if you ignore volumetric attack.

Lecture 13 · 180 minutes

Building it so the model is true

Take one abuse case and implement the control with a test that fails when the control is removed. This is the same shape as every other course's last project, aimed at an attack. The test is the evidence the model is not a PDF.

If the abuse is injection, the test sends a quote and a second statement and asserts one statement ran. If the abuse is a missing access check, the test authenticates as the other user and asserts a refusal. If the abuse is a replay, the test sends the request twice.

Do not fix the test by weakening it when it fails. A test that used to send a quote and now sends a letter is not a passing control. The software review rule is the same.

When the test passes, write the one-line control next to the abuse case on the threat-model page. The two documents agree. A model that lists a control the code does not have is a liability, not a design.

From the model to a test.

1. Pick one abuse

2. Write the attack as a test

3. Watch it fail

4. Add the control

5. Cite the test on the model

A second statement

1. The login query is built by gluing the name into SQL, as in the injection lecture.

2. The test submits a name that closes the quote and adds a condition that is always true.

3. The buggy code authenticates. The parameterized query does not. The test is the lecture's example turned into a regression.

Problem 1. The test uses a mocked database that never executes SQL. Does it catch injection?

Answer. No. The mock never concatenates. Test against a real query execution, or a library call that would reject the extra statement. A mock of the success path hides the bug.

Problem 2. The threat model lists replay protection and the code has no stored nonce. What do you change first, the model or the code?

Answer. Either add the control or remove the claim. They must agree. Leaving the claim is worse, because the next reader will trust it. Prefer adding the control if the asset needs it.

Lecture 14 · 180 minutes

Security as a property you can show

You should be able to show, for a small system: a password store that survives a database copy, an access check the UI cannot skip, a channel whose name is checked, a replay that fails, and a log that does not contain the secret. Those five are the course.

The hours are the threat-model page and the one attack test, plus the problems. The opening lectures named the mechanisms. These weeks are how they fail in a program.

When a later course, or the capstone, asks you to 'add security,' this list is the answer. A badge that says HTTPS is one item on it.

The notes are original and the attacks are the standard ones. Do not collect exploit code. Write the defensive test.

Five things you can show.

A slow salted hash

A server-side access check

A checked name on the channel

A replay that fails

A log without the secret

Remove one control

1. Delete the access check. The other-user test fails. Good: the test noticed.

2. Delete the salt and the other tests stay green. That means you have no test for equal hashes. Add one.

3. A control without a test will vanish in a refactor. The engineering course and this one are the same method.

Problem 1. Which of the five fails if two users with the same password have the same stored hash?

Answer. The password store. The salt is missing or ignored. The slow hash may still be there. Both parts are the control.

Problem 2. The channel is encrypted and the certificate name is not checked. Which item failed?

Answer. The checked name. Encryption to an unauthenticated peer is a secure channel to the attacker. The lock icon is not the check.